Stabilizing GulliView
Analysis and Recommendations

DAT 295 Autonomous and Cooperative Driving Vehicle

Lukas Klapproth: lukas.m.klapproth@campus.tu-berlin.de
Supervisor:
Xiuqi Zhang: xiuqiz@chalmers.se
Course led by:
Prof. Ellad Schiller: elad.schiller@chalmers.se

1 Introduction

1.1 What is GulliView

GulliView is a multi-camera, low-latency visual localization tool used to track WiFiBots on a test track. It is a fundamental tool enabling further research into autonomous vehicles.

Previous iteration of GulliView, had problems with the cameras being suspended and the resulting empty frames that were generated crashed the the system. Thanks to the work of Xiuqi Zhang, those issues were resolved and the system was made more resilient and is now able to handle empty frames.

Despite the improvements GulliView still suffers from instabilities which do not cause catastrophic crashes but stalls or "freezes" of the system. This Project was originally aimed on eliminating those problems, this did not work out as we discovered that the origins and mechanisms of the erroneous behavior were substantially different from those we expected in the beginning.

The most up to date version of the GulliView system can be found on the BitBucket repository https://bitbucket.org/automationarticulatedvehicles/gulliview_multithreading/src/global_coordination/, in the branch `global_coordination`. Further attempts to solve the problem can be found on the branch `as_is_branch`.

1.1.a Current Problems

The current Version of GulliView suffers from "freezes" that render the system inoperable for 10 seconds at a time and make it unsuitable for task it was designed for. We assume that these freezes are due to memory leaks in the program. We have established the critical path of this error through the program. The error is caused while checking for an available frame, in combination with a severe memory leak which then leads to the "freezes".

2 Problem Analysis

2.1 Current Problems

The current GulliView System[1] periodically stops working for individual cameras after an initial operational phase. Figure 1 shows the mean time between failures, the x-axis denotes the n-th error and the y-axis denotes the time since the last error occurred. As it is shown in Figure 1 the first error occurs after a longer initial stable phase, while the mean time between failures decreases with further errors.

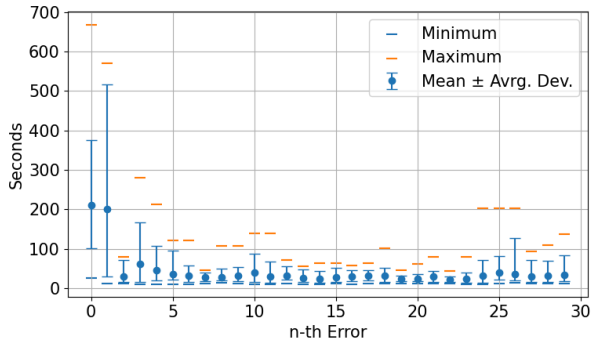

Figure 1: *mean time between failures*

An error in this context means that a camera stoped responding, and does not serve new frames. If such an error occurs the system does not respond for 10 seconds until a internal timeout in OpneCV is reached, and an empty frame is returned wich can be detected. After a an error is detected the corresponding Camera is restarted. At first a restart of a camera solves the problem for a short period, but the mean time between errors decreases to less then 10 seconds, which in effect means that the system spends more time in a "frozen" state than running.

2.2 Error Sources

The exact mechanisms of this faulty behavior are not known, but we have identified several potential flaws and identified potential sources of the problem as well as established that some formerly suspected error sources were not causing the misbehavior.

2.2.a Memory Leak

The primary suspected sources of the faulty behavior is a potential memory leak. To test that theory we have monitored the Memory consumption (in RAM) of the program with `htop` under the following settings with an expected memory usage of:

configuration	expected memory usage in GB
`3840x2160@60`	13 - 15
`3840x2160@30`	13 - 15
`1920x1080@60`	4 - 5
`1920x1080@60`	4 - 5

Table 1: *observed memory usage with different configurations*

The expected values are derived by inspecting the code base, where we found that the image buffers for the four cameras should dominate the memory usage of the program.

There is one buffer per camera of which 4 exist in this setup of GulliView. Each buffer stores 128 frames and each frame has 8 bit / 1 byte color depth (per color so 3 byte in total). With this we arrive at the following numbers:

$$\text{color depth} \cdot \text{number of pixel} \cdot \text{frames per buffer} \quad 1.$$
$$= 3 \cdot (3840 \cdot 2160) \cdot 128 \mid \text{for a 2160p resolution}$$

[1] as is on the Branch `global_coordination`, commit `8614e492a69f42e2865eb1fc0eac55bd8b6fdab0`

$$= 3 \cdot (1920 \cdot 1080) \cdot 128 \mid \text{for a 1080p resolution} \quad 1.$$

For four buffers at a resolution of 2160p we expect all four buffer to occupy around 13 GB in memory, and for a resolution of 1080p we expect the buffers to occupy around 4 GB of memory. These numbers are already rounded up to account for memory layout and some other meta data of the frames.

The actual measured memory consumption is:

configuration	physical/virtual memory in GB
`3840x2160@60`	20 / 35
`3840x2160@30`	21 / 36
`1920x1080@60`	10 / 21
`1920x1080@60`	10 / 21

Table 2: *observed memory usage with different configurations after 30 min*

The memory consumption seen in Table 2 appears to be larger than expected and we interpret it as an indication of a memory leak in the program, as there are no obvious sources for the differences in expected and observed memory use.

It is also of note that a virtual memory usage of 52 GB and physical usage of 34 GB has been observed at one point, after more then an hour of run time.

In Table 3 we have tracked the initial data consumption, a few seconds after GulliView is started and at what rate the memory consumption increase in MB per second. We have observed thath the rate of growth of memory consumption stays constant over the runtime of the program.

configuration	init physical/virtual memory in GB	growth rate physical/virtual MB/min
`3840x2160@60`	14/29	~200-230/200-230
`3840x2160@30`	14/29	~200-230/200-230
`1920x1080@60`	4/15	~200-230/200-230
`1920x1080@60`	4/15	~200-230/200-230

Table 3: *initial memory consumption and data grwoth*

2.2.b OpenCV Frame Grab Function

The source of the "freezes" was narrowed down to one specific function.

```
bool cv::VideoCapture.grab()
```
[2]

The `grab` function is supposed to test if a frame is available from the video capture object, that can then be retrieved.

The way it is supposed to work is: Every camera gets a Video capture object assigned to it. Those video capture object do interact with the cameras through the V4L2 (Video for Linux 2) backend. When the cameras are working and the object has been initialized the grab function test if a new frame is available from a camera, if this is the case the retrieve function is called which reads the frame, decodes it and stores the frame as a matrix of integers. Figure 2 a conceptional depiction of this behavior.

These "freezes" only appear in the grab function. We know that the length of a freeze is due to a timeout in a function responsible for calling the V4L2 backend, where internal timeout is triggered after exactly 10 seconds. In Figure 2 the critical path that triggers such a freeze is marked in red.

What the root cause of the freeze in this function is and why only this function is affected, is yet unknown but we think it is related to a potential memory leak described in Section 2.2.a.

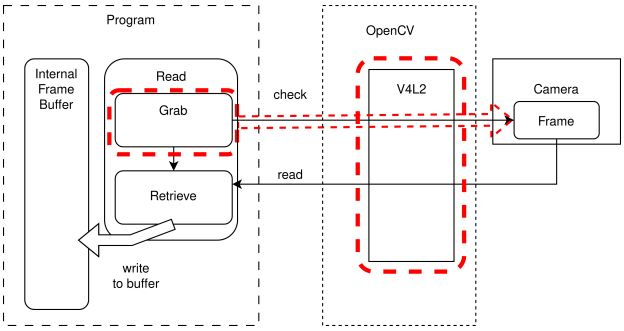

Figure 2: *conceptional critical path in GulliView*

2.3 Previously Suspected Problems

Under Previously Suspected Problems we examine problems that were suspected to cause the "freezes" in GulliView but were determined not to be the cause.

It is important to note here, that these issues may not cause the "freezes", but may still be relevant for the future development and may cause problems in the future.

2.3.a Memory Bandwidth USB Controller

A suspected issue was, that the USB controllers of the Computer could not handle the input stream of 4 cameras streaming with a resolution of 3840x2160 pixels at 60 frames per second encoded with MJPG. We suspected that the USB controller might have been overloaded and that subsequently corrupted frames were introduced and might have caused the erroneous behavior.

We have determined that this suspected issue is not a problem. When we measured the traffic over the USB ports, the highest observed traffic on any port was around 21 MBps. According to the specifications of the connected USB ports the maximum theoretical throughput is 5 GBps per port split onto two separate internal USB controller. With the known specification of the ports and adjusting for real world maximum throughput we have determined that the available bandwidth should not be saturated by the cameras, and thereby making the proposed error source implausible.

2.3.b Bad Encoding

A suspected issue was that the cameras themself send out corrupted or empty frames and thereby leading to erroneous behavior.

[2] https://docs.opencv.org/3.4/d8/dfe/classcv_1_1VideoCapture.html#ae38c2a053d39d6b20c9c649e08ff0146

We tested the output of the camera for empty frames, but such could never be observed. When testing for corrupted frames, we were able to observe some, but only sporadically and not correlated with the "freezes" of the program. We have also tested the behavior of the program when a corrupted frame is transmitted. If a frame is corrupted the program produces an empty frame, which is internally disregarded, the program stays operational as the behavior for such a case is well defined.

With our tests result we have concluded that this issue is not a problem and is most likely not the error source.

3 Recommendations

In this section we discuss future steps for this project to potentially solve the current instabilities.

3.1 Fixing the Memory Leak

A comprehensive memory analysis should be performed on GulliView. For in depth memory analysis Valgrind should be considered as a valuable tool for that task, with the important caveat that Valgrind forces application to run on a single core and single threaded.

To do an in depth memory analysis with Valgrind and the restriction of a execution on a single core, we currently see two options for working with Valgrind:

1. Modifying GulliView to only run on one camera as a single threaded application to catch any memory leaks, that are not related to the multi threaded nature of GulliView.
2. The second option consist of adding a manual task scheduler to GulliView. The idea behind that is to keep the multi threaded nature of GulliView by manually simulating multi threaded execution on one thread, with thread scheduling in user space. The programm will not execute with true parallelism but avoids/ works around the restriction of Valgrind of only one thread execution at a time.

3.1.a Excessive Matrix Copies

We recommend that the way frames are stored, copied and deleted should be reevaluated. This is not ment as a definitive statement to fundamentally change the way a frames are handled, but as a recommendation that the current implementation warrants a closer inspection as we suspect the memory leaks to have their origins in this part of the code.

Special attention should be paid to the behavior of:

- How do frame objects behave when they go out of scope, but are not explicitly deleted[3]. With an special focus on the points where these objects cross a thread boarders.
- The frame buffer for each camera, is populated with full copies of read frames. It might be advisable to only store pointer references[4] instead of full copies of the read frame objects.
- The amount of copies that are created for passing frames should be reevaluated, as this can have unnecessary performance overhead as well as potentially leading to memory leaks.

3.2 Workarounds

If the memory leak can not be identified or turns out to be not the source of the instability of GulliView, we have some recommendations that may be useful to deal with the instabilities in the short turn. The following two suggestions should not be treated as fixes for the instabilities but as short term workarounds.

3.2.a Ostrich Algorithm[5]

A potential short term solution might be to run GulliView with a resolution of 1920x1080 pixel at 60 frames per second. This does not solve the problem of the instabilities ,but we have found that in around 80% of runs the first instabilities only occur after 30 minutes or an even longer time (error free operation have been observed for up to 1 hour and 16 minutes). For short experiments using the GulliView system this might be a work around. Figure 3 shows that the mean time until the first "freeze" occurs is significantly greater than compared to the standard setup of GulliView seen in Figure 1

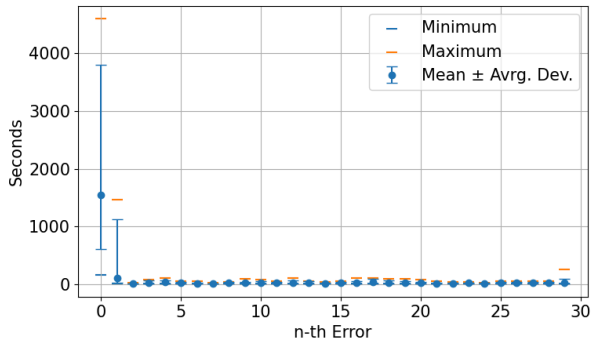

Figure 3: *mean time between failures for a resolution of 1920x1080 at 60 fps*

3.2.b Piping FFMPEG

A second workaround involves removing the use of the `bool cv::VideoCapture.grab()` function. This is achieved by taking the responsibility of retrieving frames from the camera from OpenCV and using FFMPEG instead to retrieve and decode the images from the cameras and then piping them back into GulliView[6].

[3] This might be a none issue, but my (Lukas Klapproth) knowledge of C++ deconstructors is not enough to definitively conclude that

[4] This might already be the case, depending on compiler optimization.

[5] https://en.wikipedia.org/wiki/Ostrich_algorithm

[6] This version can be found on branch: `as_is_branch`, commit: `0c7de3296a7f8104a89905c3e2e61bc0dea14e89`

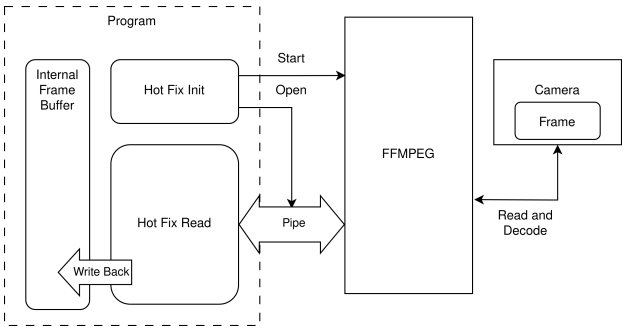

Figure 4: *schematic of the the FFMPEG work around*

The current implementation of this hot fix does not solve the issue of the memory leak but it does not suffer from freezes, even with a resolution of 3840x2160 pixels and 60 frames per second. The current implementations suffers from the drawback that it has an image latency from a few hundred milliseconds to several seconds[7] depending on the setting of Gulliview.

This work around will also stop working after 30 to 90 min depending on the setting, as the memory leak grows to large. We have found that 30 to 40 GB memory consumption in GulliView seems to be a magic number. When the threshold of 30 to 40 GB is crossed, we can observe that the leak starts to affect this work around and may cause a complete stall of the program.

4 Conclusions

We concluded that the most pressing issue of the GulliView system, as well as the source of the instabilities, is a memory leak.

A memory analysis should be performed on the GulliView system to identify the memory leak. The fix will probably include a partial rewrite of the code base of GulliView that deals with the image retrieval from the cameras, or in the worst case changes in OpenCV.

We are of the opinion that the USB controllers and their associated bandwidths are not an issue, and should even be able to handle more throughput. Empty frames and corrupted frames also do not appear to be a problem for the GulliView system.

[7] This issue might be easily resolved as this work around was only developed near the end of this project and there is still room for optimization

5 Appendix

This section includes commentary that is not directly related to GulliViews instabilities but may be useful for future work on GulliView.

5.1 OpenCV and H.264

The H.264 backend of the OpenCV version that is used in GulliView does not work. If the current OpenCV version is polled wether it supports H.264 it will report true, it is also a valid encoder option that may be set for a video capture object, but the results are garbage data and no valid frames will be produced.

The solution for this issue would most likely be to recompile OpenCV and statically link the required libraries against OpenCV and setting the correct compile flags.

5.2 Encoding Quality Camera H.264

The cameras used in the lab EDIT 5355, Elgato Facecam Pros, produce valid H.264 data streams, but the compression ratio of theses streams is worse then for the same resolution and frame rate encoded with MJPG. This indicates that the internal video compression, usig H.264, of the cameras is not very good. H.264 has a significant higher compression potential and thereby should achieve far smaller data streams. When measured the encoded video a static scene is around 5% - 10% larger when using H.264 when compared to MJPG

6 Appendix II

The second appendix expands on the FFMPEG work around. The application referenced here is on the branch `as_is_branch`, commit: `0c7de3296a7f8104a89905c3e2e61bc0dea14e89`.

6.1 Motivation

This work around was developed for two distinct reasons.

1. To be able, to decode H.264 video streams.
2. To avoid OpenCV retrieving the frames from the cameras, as it is known, that the freezes appear in this part of the code.

We decided to use FFMPEG as it is a very robust and flexible application with integrated support for most codecs and debugging options.

6.2 Setup GulliView

At the initializing process of GulliView we create an array containing 4 FILE pointers, which we populate with pointer to streams.

Please note that the code is written in C-style C++, this is not a specific design choice, and more a matter of practicality as the developer[8] was proficient in C and it made the development for this prototype easier. For potential future work it may be advisable to rewrite this in proper and idiomatic C++.

The array containing the pointers to the streams is a global variable with no access protection. To deal with the multi threaded nature of GulliView, every pointer is associated with one camera, so thereby only the producer thread for the i-th camera may access the i-th FILE pointer in the array and may read a frame from the stream.

6.3 Reading from Stream

FFMPEG is setup to read frames from the camera and decode the frames to a raw video format, where a pixel is represented by a set of 3 bytes (brg), which is then written to to stdout. When GulliView reads from the stream it writes the data to the data array of a frame object, as the data buffer of a frame object is just a `uint_8` array under the hood. All further processing of frames in GulliView was left unchanged.

6.4 Setup of FFMPEG

FFMPEG is called with the following arguments:

- -hide_banner
  hides FFMPEG startup text
- -f v4l2
  sets the backend to video for linux 2, to be able to talk to the cameras
- -framerate FPS
  sets the frame rate
- -video_size `XxY`
  set the expected video input resolution and sets the output resolution of the cameras over v4l2
- -input_format mjpeg
  set the expected video input format and sets the output format of the cameras over v4l2
- -i %s
  is the file path to the cameras
- -c:v rawvideo
  sets the internal decoder of FFMPEG from format to raw video
- -pix_fmt bgr24
  sets the format of the raw video, in this case bgr with 1 byte per color
- -f image2pipe -
  write the raw video to stdout

[8] Lukas Klapproth, lukas.m.klapproth@campus.tu-berlin.de